

방학 - 5주차

날짜 2026년 8월 23일

속도 개선을 위한 좇뱅이

실험 1. 채널 축소 — 어디까지 줄일 수 있나

87채널에서 시작해 여러 구성을 학습·측정했어.

구성	지연	macro-F1	DEMAG 평균	판정
15ch	9.5 ms	0.870	0.781	최전선 (하한 앵커)
48ch	20.2 ms	0.965	0.935	최전선
51ch 절대축	21.5 ms	0.964	0.928	48ch에 지배
81ch	33.4 ms	0.972	0.951	최전선
87ch	35.4 ms	0.973	0.955	최전선

얻은 것: 절대축 하나가 도약의 대부분을 만들었어. 15 → 48이 F1 +0.095인데 48 → 81은 +0.007. 그리고 51채널이 48채널보다 성능이 낮은 유일한 역전 사례가 나왔는데,

`band_ratio_7` 이 float16 언더플로우(5.91%)로 죽은 컬럼이었기 때문이야. **경량화 효과가 아니라 데이터 품질 결함을 잡은 거고.**

실험 2. 커널 축소 — 정보를 안 버리고 줄일 수 있나

87채널 고정, `NUM_KERNELS` 만 변경.

커널	지연	모델 크기	DEMAG 평균	게이트
9,996	35.4 ms	0.69 MB	0.9551	기준
4,956	34.3 ms (-3%)	0.36 MB	0.9478	탈락
2,436	31.8 ms (-10%)	0.20 MB	0.9476	탈락
1,176	17.7 ms (-50%)	0.10 MB	0.9225	탈락

가설이 깨졌어. "지연의 97%가 transform이니 커널을 줄이면 지연도 준다"고 봤는데, 모델 크기는 커널 수에 정확히 비례한 반면 **지연은 안 따라왔어.** 커널 절반에 -3%. 87채널 × 2000프레임을 훑는 비용이 커널 개수와 무관하게 깔려 있었던 거야. 모델 크기가 대조군 역할을 해서 이게 추측이 아니라 증명이 됐고.

그리고 4,956은 2,436에 모든 축에서 지배당해 — 성능 같고 모델 두 배, 3ms 느려. 51채널과 같은 패턴이 커널 축에서 반복됐어.

실험 3. 축 교체 — 차수축이 더 나은 표현인가

51채널 유지, 절대축 → 차수축 통째 교체.

	51ch 절대축	51ch 차수축	87ch 둘 다
macro-F1	0.9635	0.9562	0.9729
KONA 미검출	372 (11.4%)	341 (10.5%)	199 (6.1%)
KONA 오경보	290 (9.4%)	400 (12.9%)	245 (7.9%)

두 축이 반대 방향으로 틀려. 차수축은 미검출이 적고 오경보가 많고, 절대축은 반대. 합치니 둘 다 동시에 줄어 — 임계값 이동으로는 불가능한 결과라, 87채널의 이득이 채널 수가 아니라 정보 상보성에서 왔다는 직접 증거야.

최종 후보

속도 축에서 고를 이유가 사라졌어. i9 기준 가장 긴 zsplit 조각이 87채널로도 2.6초, 예산 10초. 커널로도 못 줄이고, 채널로 줄이면 정확도를 지불해야 하는데 줄여야 할 이유 자체가 없어.

그래서 남은 후보는 둘:

87ch (기본값) — 0.9729 / DEMAG 0.9551 / 35.4ms / 0.69MB. 정확도 축 전부 1위. 세 실험 다 이걸 방어하는 쪽으로 결론이 났어.

48ch (예비) — 0.965 / DEMAG 0.935 / 20.2ms. 지연이 절반. Pi 4나 차량 진단기에서 87ch가 빠듯할 때만 꺼내는 카드고, 그 전엔 쓸 이유가 없어.

2,436커널 은 후보가 아니라 보험이야. 저장 공간이 제약이 되면 (모델 0.20MB, -71%) 그때 유일한 선택지고, 지금은 아니야.

남은 것

Pi 4 실측 하나. "지연이 제약이 아니다"가 지금 전체 결론을 떠받치고 있는데, 근거가 i9 16코어 측정뿐이야. Pi에서 87ch가 통과하면 최종 확정이고, 빠듯하면 48ch로 내려가면서 실험 1의 파레토 그래프가 판정 근거로 되살아나. 후보가 2개로 좁혀졌으니 측정도 가볍고.

그리고 서사를 이렇게 잡는 게 정직해. 경량화는 데이터 표현(1.15TB → 107GB, 90.7%)과 모델 선택(YOLOv11-Seg → MiniRocket+LR) 단계에서 이미 달성됐어. 실험 1~3은 그게 충분한지 검증한 거고, 결과는 "청크당 35ms · 모델 0.69MB로 추가 축소가 불필요"야. 여

기에 "커널 수는 모델 크기의 손잡이지 자연의 손잡이가 아니다" 라는 발견이 부수적으로 따라붙고. 음성 결과지만 측정으로 뒷받침된 거라 발표에서 값어치가 있어.

노무현파이 실험 결과

• 실시간 판정: 전부 통과, 그것도 여유롭게

체크 = 10초인데 가장 무거운 모델이 455ms입니다.

모델	자연시간	CPU	듀티	여유율	F1
87ch k1250	226.9 ms	2.3%	44배	0.959	
87ch k2500	412.0 ms	4.1%	24배	0.959	
87ch k5000	446.6 ms	4.5%	22배	0.971	
87ch k10000	455.0 ms	4.5%	22배	0.973	

가장 정확한 모델을 그냥 쓰시면 됩니다. 10초마다 455ms만 쓰고 9.545ms(95.5%)는 놓고 있습니다. 코어 하나로 이렇게 되고, 나머지 3코어는 아예 손도 안 땁니다.

그래서 결론이 뒤집힙니다

지금까지 커널 축소-커널 축소를 "몇지 배포를 위한 경량화"로 논의해왔는데, 이 시나리오에서는 경량화할 이유가 없습니다. k1250이 2배 빠르지만 아차치 둘 다 여유율이 20배를 넘습니다. 44배 여유와 22배 여유의 실질 차이는 없고, F1 0.014는 실제로 읽는 것입니다. k10000(F1 0.973) 채택이 맞습니다.

발표 문장으로 쓰신다면:

라즈베리파이 4에서 10초 체크당 추론 455 ms, 듀티 사이클 4.5%로 실시간 요건을 22배 여유로 충족, 단일 코어만 사용하며 스토리밍 없음(최고 43.8°C, 상한 80°C).

경량화 성과는 다른 축에서 말씀하세요

"몇지에서 못 돌아가서 줄였다"가 아니라, 이미 하산 작업들이 더 실효력 있습니다:

- PNG → 4피치 CSV로 약 12배 용량 감소 (600GB 데이터 피아프라인)
- 학습 시간 28시간 → 1시간 52분
- 모델 2.06 MB, 적체 0.25초, RSS 193 MB — 4GB 피아에서 5% 미만
- 커널 스캔으로 정확도-속도 곡선을 살펴서 "중요 필요 없음"을 데이터로 확인

마지막 항목이 특히 좋습니다. 커널을 반으로 줄여도 안 빨라지는 이유(dilation 상한 32에서 포화)를 PC와 파이 양쪽에서 오차 2% 이내로 제한했으니, 근거 없이 "경량화했다"는 것보다 훨씬 단단합니다.